Part 1. 음악 AI 수학적 심화

목차

1. 개요
2. 생성 모델의 수학적 기초
3. VAE (Variational Autoencoder)
4. 뉴럴 오디오 코덱
5. Diffusion Models
6. Latent Diffusion Model
7. DiffRhythm 심화 분석
8. PyTorch 구현 예시
9. 하이퍼파라미터 가이드
10. 참고 문헌

1. 개요

이 문서는 **음악 AI의 핵심 생성 모델**에 대한 수학적 원리와 구현을 심층 분석합니다.

1.1 Part 1에서 다루는 내용

- **VAE**: 오디오 압축의 수학적 원리
- **EnCodec/SoundStream**: 뉴럴 코덱 아키텍처
- **Diffusion Models**: DDPM, DDIM, CFG
- **Latent Diffusion**: 효율적 생성 모델
- **DiffRhythm**: 가사 정렬 음악 생성
- **PyTorch 구현**: 실제 작동 코드

1.2 사전 지식 요구사항

- 확률론: 확률 분포, KL Divergence, 베이즈 정리
- 딥러닝: Autoencoder, CNN, Transformer
- 신호처리: STFT, Mel-Spectrogram
- PyTorch 기초: 자동미분, 최적화

1.3 주요 모델 비교

모델	압축 방법	생성 방법	대표 응용
VAE	확률적 인코딩	잠재 변수 샘플링	Stable Audio, DiffRhythm
EnCodec	RVQ 이산화	토큰 시퀀스	MusicGen, AudioGen
Diffusion	-	반복 디노이징	Stable Diffusion, DiffRhythm
LDM	VAE + Diffusion	잠재 공간 디노이징	Stable Audio, DiffRhythm

2. 생성 모델의 수학적 기초

2.1 확률론적 관점

생성 모델의 궁극적 목표는 **데이터의 실제 확률 분포** $p_{\text{data}}(x)$**를 학습**하는 것이다.

이 분포를 알면:

1. **샘플링**: $x \sim p_{\text{data}}(x)$로 새로운 데이터 생성
2. **밀도 추정**: 주어진 x가 얼마나 "그럴듯한지" $p_{\text{data}}(x)$로 평가
3. **조건부 생성**: $p(x \mid c)$로 조건 c에 맞는 데이터 생성

그러나 $p_{\text{data}}(x)$는 직접 알 수 없다. 우리가 가진 것은 데이터셋 $\mathcal{D} = \{x_1, x_2, \ldots, x_N\}$뿐이다.

2.2 최대 우도 추정 (MLE)

파라미터 θ를 가진 모델 $p_\theta(x)$로 $p_{\text{data}}(x)$를 근사한다.

$$\theta^* = \arg\max_\theta \prod_{i=1}^{N} p_\theta(x_i)$$

로그를 취하면 (곱을 합으로):

$$\theta^* = \arg\max_\theta \sum_{i=1}^{N} \log p_\theta(x_i) = \arg\max_\theta \ \mathbb{E}_{x\sim p_{\text{data}}}[\log p_\theta(x)]$$

2.3 KL Divergence와의 관계

MLE는 **KL divergence를 최소화**하는 것과 동치:

$$D_{KL}(p_{\text{data}} \| p_\theta) = \mathbb{E}_{x\sim p_{\text{data}}}\left[\log \frac{p_{\text{data}}(x)}{p_\theta(x)}\right]$$

$$= \mathbb{E}_{x\sim p_{\text{data}}}[\log p_{\text{data}}(x)] - \mathbb{E}_{x\sim p_{\text{data}}}[\log p_\theta(x)]$$

$$= \underbrace{H(p_{\text{data}})}_{\text{상수}} - \mathbb{E}_{x\sim p_{\text{data}}}[\log p_\theta(x)]$$

첫 번째 항은 θ와 무관한 상수이므로:

$$\min_\theta D_{KL} \Leftrightarrow \max_\theta \mathbb{E}[\log p_\theta(x)] \Leftrightarrow \text{MLE}$$

2.4 생성 모델의 분류

```
생성 모델
├── Explicit Density (확률 분포 명시적 학습)
│   ├── Tractable Density (계산 가능)
│   │   ├── Autoregressive Models (PixelCNN, WaveNet, GPT)
│   │   └── Normalizing Flows (RealNVP, Glow)
│   └── Approximate Density (근사적 계산)
│       ├── VAE (Variational Autoencoder)
│       └── Diffusion Models (DDPM, Score Matching)
└── Implicit Density (확률 분포 암묵적 학습)
    └── GAN (Generative Adversarial Networks)
```

음악 AI에서의 활용:

- **Autoregressive**: MusicGen, Jukebox (순차적 토큰 생성)
- **VAE**: Stable Audio VAE, DiffRhythm VAE (잠재 공간 압축)
- **Diffusion**: DiffRhythm DiT, AudioLDM (잠재 공간 디노이징)

---

3. VAE (Variational Autoencoder)

VAE는 음악 AI의 핵심 컴포넌트다. Stable Audio, DiffRhythm 모두 VAE로 오디오를 압축한 뒤 Diffusion을 수행한다.

3.1 잠재 변수 모델

핵심 가정: 관측 데이터 x는 **관측되지 않는 잠재 변수 z**로부터 생성된다.

$$z \sim p(z) \quad \text{(Prior: 잠재 변수의 사전 분포)}$$

$$x \sim p_\theta(x \mid z) \quad \text{(Likelihood: 잠재 변수로부터 데이터 생성)}$$

Marginal likelihood (데이터의 확률):

$$p_\theta(x) = \int p_\theta(x \mid z) p(z)\, dz$$

문제: 이 적분은 일반적으로 **계산 불가능(intractable)**하다.

3.2 True Posterior의 계산 불가능성

Bayes' rule:

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z) \cdot p(z)}{p_\theta(x)}$$

분모 $p_\theta(x)$를 계산하려면:

$$p_\theta(x) = \int p_\theta(x \mid z) \cdot p(z)\, dz$$

구체적 예시: $z \in \mathbb{R}^{64}$이고, 각 차원을 100개 구간으로 이산화하면:

$$\text{총 경우의 수} = 100^{64} = 10^{128}$$

우주의 원자 수($\sim 10^{80}$)보다 많다!

3.3 Variational Inference

해결책: True posterior $p_\theta(z \mid x)$를 다루기 쉬운 분포 $q_\phi(z \mid x)$로 **근사**:

$$q_\phi(z \mid x) \approx p_\theta(z \mid x)$$

VAE에서는 가우시안으로 가정:

$$q_\phi(z \mid x) = \mathcal{N}(z; \mu_\phi(x), \sigma_\phi^2(x) I)$$

여기서 $\mu_\phi(x)$와 $\sigma_\phi(x)$는 인코더 네트워크의 출력.

3.4 ELBO (Evidence Lower Bound) — 완전 유도

Step 1: 출발점

$$\log p_\theta(x) = \log p_\theta(x) \cdot \int q_\phi(z \mid x)\, dz = \int q_\phi(z \mid x) \cdot \log p_\theta(x)\, dz$$

Step 2: Bayes' rule 대입

$$p_\theta(x) = \frac{p_\theta(x \mid z) \cdot p(z)}{p_\theta(z \mid x)}$$

대입:

$$\log p_\theta(x) = \int q_\phi(z \mid x) \cdot \log \frac{p_\theta(x \mid z) \cdot p(z)}{p_\theta(z \mid x)}\, dz$$

Step 3: 분모·분자에 $q_\phi(z \mid x)$ 곱하기

$$= \int q_\phi(z \mid x) \cdot \log \frac{p_\theta(x \mid z) \cdot p(z) \cdot q_\phi(z \mid x)}{p_\theta(z \mid x) \cdot q_\phi(z \mid x)}\, dz$$

Step 4: 로그의 곱을 합으로 분리

$$= \int q_\phi(z \mid x) \cdot \log p_\theta(x \mid z)\, dz - \int q_\phi(z \mid x) \cdot \log \frac{q_\phi(z \mid x)}{p(z)}\, dz + \int q_\phi(z \mid x) \cdot \log \frac{q_\phi(z \mid x)}{p_\theta(z \mid x)}\, dz$$

Step 5: 각 항의 정체

항 (A): Reconstruction Term

$$\mathbb{E}_{z \sim q_\phi(z|x)}[\log p_\theta(x \mid z)]$$

해석: 인코더가 압축한 z로부터 디코더가 원본 x를 얼마나 잘 복원하는가.

항 (B): KL Divergence (인코더 ↔ Prior)

$$D_{KL}(q_\phi(z \mid x) \| p(z))$$

해석: 인코더가 근사한 분포가 사전 분포(표준 정규분포)와 얼마나 다른가.

항 (C): KL Divergence (인코더 ↔ True Posterior)

$$D_{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x))$$

이 항은 계산 불가능하지만 **항상** ≥ 0.

Step 6: 최종 ELBO

$$\log p_\theta(x) = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] - D_{KL}(q_\phi(z \mid x) \| p(z))}_{\text{ELBO}} + \underbrace{D_{KL}(q_\phi \| p_\theta(z \mid x))}_{\geq 0}$$

따라서:

$$\boxed{\log p_\theta(x) \geq \text{ELBO} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - D_{KL}(q_\phi(z \mid x) \| p(z))}$$

핵심 통찰:

1. ELBO를 maximize하면 $\log p_\theta(x)$도 증가
2. q_ϕ가 $p_\theta(z \mid x)$에 가까워질수록 ELBO가 tight해짐

3.5 두 가우시안 간 KL Divergence — 완전 유도

두 분포:

- $q = \mathcal{N}(\mu, \sigma^2)$: 인코더 출력
- $p = \mathcal{N}(0, 1)$: 표준 정규분포

$$D_{KL}(q \| p) = \int q(z) \log \frac{q(z)}{p(z)} \, dz$$

가우시안 PDF

$$q(z) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(z-\mu)^2}{2\sigma^2}\right)$$

$$p(z) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{z^2}{2}\right)$$

로그 비율

$$\log \frac{q(z)}{p(z)} = -\frac{1}{2} \log \sigma^2 - \frac{(z-\mu)^2}{2\sigma^2} + \frac{z^2}{2}$$

기댓값 계산

항 1: $\mathbb{E}_q\left[-\frac{1}{2} \log \sigma^2\right] = -\frac{1}{2} \log \sigma^2$

항 2: $\mathbb{E}_q\left[-\frac{(z-\mu)^2}{2\sigma^2}\right] = -\frac{\sigma^2}{2\sigma^2} = -\frac{1}{2}$

항 3: $\mathbb{E}_q\left[\frac{z^2}{2}\right] = \frac{\sigma^2+\mu^2}{2}$

합산

$$D_{KL}(q\|p) = -\frac{1}{2}\log\sigma^2 - \frac{1}{2} + \frac{\sigma^2+\mu^2}{2}$$
$$= -\frac{1}{2}\left(1 + \log\sigma^2 - \mu^2 - \sigma^2\right)$$

d차원으로 확장:

$$\boxed{D_{KL}(q_\phi(z \mid x)\|p(z)) = -\frac{1}{2}\sum_{j=1}^{d}\left(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2\right)}$$

3.6 Reparameterization Trick

문제: 샘플링은 미분 불가능

$$z \sim \mathcal{N}(\mu, \sigma^2)$$

"샘플링"은 확률적 연산 → 그래디언트 통과 불가.

해결: 확률성을 외부로 분리

$$\epsilon \sim \mathcal{N}(0, I)$$
$$z = \mu + \sigma \odot \epsilon$$

이제 z는 μ, σ의 **결정적 함수**:

$$\frac{\partial z}{\partial \mu} = I, \quad \frac{\partial z}{\partial \sigma} = \mathrm{diag}(\epsilon)$$

왜 동치인가?

확률변수 $\epsilon \sim \mathcal{N}(0,1)$에 대해:

- $\sigma \cdot \epsilon \sim \mathcal{N}(0, \sigma^2)$
- $\mu + \sigma \cdot \epsilon \sim \mathcal{N}(\mu, \sigma^2)$

3.7 VAE 손실 함수

Reconstruction Term

Gaussian Decoder (MSE):

$$\mathcal{L}_{\text{recon}} = \frac{1}{2\sigma_{\text{dec}}^2}\|x - \hat{x}\|_2^2$$

$\sigma_{\text{dec}} = 1$로 고정하면 MSE.

오디오 VAE (Multi-Resolution STFT):

$$\mathcal{L}_{\text{recon}} = \sum_{k=1}^{K}\left(\||S_k(x)| - |S_k(\hat{x})|\|_F + \|\log|S_k(x)| - \log|S_k(\hat{x})|\|_1\right)$$

여기서:

- S_k: k번째 해상도의 STFT
- $|\cdot|$: magnitude
- $\|\cdot\|_F$: Frobenius norm

다중 해상도의 이유: 고주파(작은 FFT)와 저주파(큰 FFT)를 동시에 포착.

최종 VAE 손실

$$\boxed{\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}} + \beta \cdot D_{KL}(q_\phi(z \mid x) \| p(z))}$$

β 설정:

- $\beta = 1$: 원본 VAE
- $\beta < 1$: 복원 품질 우선 (오디오)
- $\beta > 1$: 잠재 공간 구조 우선 (β-VAE)

3.8 Posterior Collapse

현상

$$D_{KL}(q_\phi(z \mid x) \| p(z)) \to 0$$

인코더가 모든 x에 대해 $q_\phi(z \mid x) \approx \mathcal{N}(0, I)$를 출력.

원인

디코더가 **너무 강력**하면 z 없이도 복원 가능:

$$p_\theta(x \mid z) = \prod_t p_\theta(x_t \mid x_{<t}, z) \approx \prod_t p_\theta(x_t \mid x_{<t})$$

해결책

1. KL Annealing:

$$\beta(t) = \min(1, t/T_{\text{anneal}})$$

2. Free Bits:

$$\tilde{D}_{KL} = \max(D_{KL}, \lambda)$$

3. 디코더 약화: Autoregressive 대신 CNN.

---

4. 뉴럴 오디오 코덱

4.1 왜 뉴럴 코덱인가?

특성	전통 코덱 (MP3, AAC)	뉴럴 코덱 (EnCodec)
설계	수작업 규칙	데이터 학습
비트레이트	128kbps+	**1.5~24kbps**

이산 표현	없음	**있음** (토큰)
LLM 호환	불가	**가능**

4.2 EnCodec 아키텍처

```
[인코더]
24kHz 오디오 x ∈ ℝ^T
    ↓ Conv1D (C=32, k=7, s=1)
    ↓ ConvBlock (s=2, C=64)   → T/2
    ↓ ConvBlock (s=4, C=128)  → T/8
    ↓ ConvBlock (s=5, C=256)  → T/40
    ↓ ConvBlock (s=8, C=512)  → T/320
    ↓ LSTM (2 layers)
    ↓ Conv1D (C=128, k=7, s=1)
    ↓ z ∈ ℝ^{T/320 × 128}

[RVQ (Residual Vector Quantization)]
z → 8단 양자화 → 8개 코드북 인덱스/프레임

[디코더]
양자화된 ẑ
    ↓ Conv1D (C=512)
    ↓ LSTM (2 layers)
    ↓ ConvTransposeBlock (s=8, C=256)
    ↓ ConvTransposeBlock (s=5, C=128)
    ↓ ConvTransposeBlock (s=4, C=64)
    ↓ ConvTransposeBlock (s=2, C=32)
    ↓ Conv1D (C=1)
    ↓ x̂ ∈ ℝ^T
```

압축률:

- 24kHz → 75Hz (320배 시간 압축)
- 128차원 → 8×10bit = 80bit/프레임
- 비트레이트: 75 × 80 = **6kbps** (vs MP3 128kbps → **21배**)

4.3 Residual Vector Quantization (RVQ)

단일 VQ의 한계

$$\hat{z} = \arg\min_{c \in \mathcal{C}} \|z - c\|_2$$

문제: 코드북 크기 K를 키우면 탐색 비용 $O(K)$ 증가.

RVQ 알고리즘

```
z₀ = z
for q = 1 to Q:
    ẑ^(q) = argmin_{c ∈ C_q} ‖z_{q-1} - c‖²
    z_q = z_{q-1} - ẑ^(q)
end
ẑ = Σ_{q=1}^{Q} ẑ^(q)
```

장점:

- 탐색: $O(Q \cdot K)$ vs $O(K^Q)$
- 점진적 정밀도 향상

RVQ 학습

Straight-Through Estimator (STE):

$$\text{Forward: } \hat{z} = \text{quantize}(z)$$

$$\text{Backward: } \frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial \hat{z}}$$

Codebook Loss:

$$\mathcal{L}_{\text{codebook}} = \sum_{q=1}^{Q} \|\text{sg}[r^{(q-1)}] - \hat{z}^{(q)}\|_2^2$$

Commitment Loss:

$$\mathcal{L}_{\text{commit}} = \beta \sum_{q=1}^{Q} \|r^{(q-1)} - \text{sg}[\hat{z}^{(q)}]\|_2^2$$

4.4 Discriminator와 GAN 학습

Multi-Scale Discriminator (MSD)

$$D_{\text{MSD}} = \{D_1, D_2, D_3\}$$

- D_1: 원본 해상도
- D_2: 2배 다운샘플
- D_3: 4배 다운샘플

Multi-Period Discriminator (MPD)

주기적 패턴 평가:

$$\text{reshape}(x, p) = \text{2D array of period } p$$

Adversarial Loss

$$\mathcal{L}_{\text{adv}}^{(G)} = \mathbb{E}\left[(D(\hat{x}) - 1)^2\right]$$

$$\mathcal{L}_{\text{adv}}^{(D)} = \mathbb{E}\left[(D(x) - 1)^2 + D(\hat{x})^2\right]$$

Feature Matching Loss

$$\mathcal{L}_{\text{feat}} = \sum_l \|D^{(l)}(x) - D^{(l)}(\hat{x})\|_1$$

4.5 EnCodec 최종 손실

$$\mathcal{L} = \lambda_t \mathcal{L}_{\text{time}} + \lambda_s \mathcal{L}_{\text{STFT}} + \lambda_g \mathcal{L}_{\text{adv}}^{(G)} + \lambda_f \mathcal{L}_{\text{feat}} + \lambda_c (\mathcal{L}_{\text{codebook}} + \mathcal{L}_{\text{commit}})$$

일반 설정: $\lambda_t = 0.1$, $\lambda_s = 1$, $\lambda_g = 3$, $\lambda_f = 3$, $\lambda_c = 1$

---

5. Diffusion Models

5.1 Score Matching 관점

Score function:

$$s_\theta(x, t) = \nabla_x \log p_t(x)$$

데이터 분포의 로그 그래디언트 → **밀도가 높은 방향**.

Denoising Score Matching:

$$s_\theta(x_t, t) \approx -\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \epsilon$$

Score matching과 ϵ-prediction은 동치.

5.2 DDPM (Denoising Diffusion Probabilistic Model)

Forward Process

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

Linear Schedule:

$$\beta_t = \beta_{\min} + \frac{t-1}{T-1}(\beta_{\max} - \beta_{\min})$$

일반적: $\beta_{\min} = 10^{-4}$, $\beta_{\max} = 0.02$

Cosine Schedule (improved):

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos\left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2}\right)^2$$

한 번에 t 시점으로

$\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$:

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I)$$

샘플링:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Reverse Process

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$$

μ_θ **계산**:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$$

DDPM 학습 알고리즘

```
repeat:
    x₀ ~ q(x₀)
    t ~ Uniform{1, ..., T}
    ε ~ N(0, I)
    x_t = √ᾱ_t · x₀ + √(1-ᾱ_t) · ε
    Loss = ‖ε - ε_θ(x_t, t)‖²
    θ ← θ - η▽_θ Loss
until converged
```

DDPM 샘플링

```
x_T ~ N(0, I)
for t = T, T-1, ..., 1:
    z ~ N(0, I) if t > 1 else z = 0
    ε̂= ε_θ(x_t, t)
    x_{t-1} = (1/√α_t)(x_t - (β_t/√(1-ᾱ_t))ε̂) + σ_t · z
end
return x₀
```

5.3 DDIM (빠른 샘플링)

핵심 아이디어

DDPM은 확률적(stochastic), DDIM은 결정적(deterministic):

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left(\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } x_0} + \sqrt{1 - \bar{\alpha}_{t-1}} \cdot \epsilon_\theta(x_t, t)$$

가속 샘플링

부분 시퀀스 $\{\tau_1, \ldots, \tau_S\}$ 사용:

```
x_T ~ N(0, I)
for i = S, S-1, ..., 1:
    t = τ_i, s = τ_{i-1}
    ε̂= ε_θ(x_t, t)
    x_s = √ᾱ_s · (x_t - √(1-ᾱ_t)·ε̂) / √ᾱ_t + √(1-ᾱ_s) · ε̂
end
```

효과: 1000 스텝 → 50 스텝 (**20배 가속**)

5.4 Classifier-Free Guidance (CFG)

조건부 score와 무조건부 score의 외삽:

$$\tilde{\epsilon}_\theta(x_t, t, c) = \epsilon_\theta(x_t, t, \varnothing) + w \cdot (\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing))$$

- $w = 1$: 순수 조건부
- $w > 1$: 조건을 **더 강하게** (일반적 $w = 3 \sim 7$)
- $w = 0$: 무조건부

학습 시 무조건부 예측

```
if random() < 0.1:
    c = ∅  # 10% 확률로 조건 드롭
Loss = ‖ε - ε_θ(x_t, t, c)‖²
```

수학적 해석

Score function 관점:

$$\tilde{s}(x_t, c) = \nabla_x \log p(x_t) + w \cdot \nabla_x \log p(c \mid x_t)$$

$w > 1$이면 조건 방향을 더 강하게 따름.

---

6. Latent Diffusion Model

6.1 왜 잠재 공간인가?

픽셀/파형 공간의 문제:

- 오디오 3분 = 8백만 샘플
- 이미지 512×512 = 78만 픽셀
- 계산 비용 막대

잠재 공간의 장점:

- VAE로 100~1000배 압축
- 압축된 공간에서 Diffusion → **효율적**
- 고수준 특징 → **구조적 일관성**

6.2 2-Stage 파이프라인

```
[Stage 1: VAE (오프라인)]
오디오 x ∈ ℝ^T → 인코더 E → z ∈ ℝ^{T'×d} → 디코더 D → x̂

[Stage 2: Diffusion]
노이즈 z_T ~ N(0,I) → 반복 디노이징 → z_0 → 디코더 D → 생성 오디오
```

핵심: Stage 1 VAE는 고정(frozen).

6.3 Diffusion Transformer (DiT)

입력 패치화

잠재 표현 $z \in \mathbb{R}^{T' \times d}$를 패치로 분할:

$$\text{패치 수 } N = T'/p$$

$$h_i^{(0)} = W_{\text{embed}} \cdot \text{flatten}(\text{patch}_i) + \text{PE}_i$$

DiT Block

```
Input: h^{(l)} ∈ ℝ^{N×D}

1. Layer Norm + adaLN:
   h' = LN(h^{(l)}) ⊙ (1 + γ₁) + β₁

2. Multi-Head Self-Attention:
   h'' = h^{(l)} + α₁ ⊙ Attention(h')

3. Layer Norm + adaLN:
   h''' = LN(h'') ⊙ (1 + γ₂) + β₂

4. FFN:
   h^{(l+1)} = h'' + α₂ ⊙ FFN(h''')
```

adaLN (Adaptive Layer Normalization)

$$[\gamma_1, \beta_1, \alpha_1, \gamma_2, \beta_2, \alpha_2] = \text{MLP}(\text{SiLU}(\text{Embed}(t) + \text{Embed}(c)))$$

타임스텝 t와 조건 c로부터 변조 파라미터 생성.

SiLU (Swish): $\text{SiLU}(x) = x \cdot \sigma(x)$

Zero-initialized Final Layer

$$W_{\text{final}} = 0$$

학습 초기 항등 함수 → 학습 안정성.

6.4 Cross-Attention for Conditioning

텍스트 조건 주입:

$$\text{CrossAttn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- $Q = hW_Q$: 잠재 표현
- $K = cW_K$, $V = cW_V$: 텍스트 임베딩

---

7. DiffRhythm 심화 분석

7.1 Sentence-Level Lyrics Alignment

문제

가사 $(l_1, \ldots, l_S)$와 시간 구간 정렬.

정밀 정렬의 어려움:

- 음절 단위는 비용 크고 오류 발생
- 대규모 데이터 확보 어려움

해결책

문장 단위 약한 정렬:

$$\text{구간 } i : [t_i^{\text{start}}, t_i^{\text{end}}] \leftrightarrow l_i$$

Cross-Attention 구현

가사 문장 임베딩:

$$c_i = \text{T5Encoder}(l_i) \in \mathbb{R}^{L_i \times d_c}$$

해당 시간 구간 패치들이 c_i와 Cross-Attention:

$$h'_{[t_i]} = h_{[t_i]} + \text{CrossAttn}(h_{[t_i]}, c_i, c_i)$$

효과: 각 구간이 해당 가사 정보 참조 → 보컬 생성.

7.2 Non-Autoregressive 생성

장점

속도:

$$\text{Autoregressive} : O(T) \text{ 순차}$$

$$\text{Non-Autoregressive} : O(S) \text{ 순차}, \ S \approx \text{20-50}$$

4분 45초 곡: $T \sim 100{,}000$ vs $S = 50$

전역 일관성: 전체 시퀀스 동시 갱신 → 곡 구조 고려.

단점

세밀한 제어 어려움: 위치별 정밀 제어 불가.

데이터 요구량: 전역 구조 학습 필요.

7.3 Stable Audio VAE 호환성

동일한 잠재 공간:

- 잠재 차원: 64
- 시간 압축: ~512x
- 주파수: 44.1kHz → ~86Hz

호환성 장점:

1. Stable Audio VAE 재사용
2. 기존 프레임워크 통합
3. 커뮤니티 개선 즉시 적용

---

8. PyTorch 구현 예시

8.1 VAE 인코더/디코더

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class VAEEncoder(nn.Module):
    def __init__(self, in_channels=1, latent_dim=64,
                 hidden_dims=[32, 64, 128, 256]):
        super().__init__()

        modules = []
        for h_dim in hidden_dims:
            modules.append(
                nn.Sequential(
                    nn.Conv1d(in_channels, h_dim, kernel_size=4,
                              stride=2, padding=1),
                    nn.BatchNorm1d(h_dim),
                    nn.LeakyReLU()
                )
            )
            in_channels = h_dim

        self.encoder = nn.Sequential(*modules)
        self.fc_mu = nn.Linear(hidden_dims[-1], latent_dim)
        self.fc_logvar = nn.Linear(hidden_dims[-1], latent_dim)

    def forward(self, x):
        # x: (B, 1, T)
        h = self.encoder(x)  # (B, 256, T')
        h = h.mean(dim=-1)   # (B, 256)

        mu = self.fc_mu(h)
        logvar = self.fc_logvar(h)

        return mu, logvar

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std


class VAEDecoder(nn.Module):
    def __init__(self, latent_dim=64, out_channels=1,
                 hidden_dims=[256, 128, 64, 32]):
        super().__init__()

        self.fc = nn.Linear(latent_dim, hidden_dims[0] * 4)

        modules = []
        for i in range(len(hidden_dims) - 1):
            modules.append(
                nn.Sequential(
                    nn.ConvTranspose1d(hidden_dims[i], hidden_dims[i+1],
                                       kernel_size=4, stride=2, padding=1),
                    nn.BatchNorm1d(hidden_dims[i+1]),
                    nn.LeakyReLU()
                )
            `
```

```
        )

        self.decoder = nn.Sequential(*modules)
        self.final = nn.ConvTranspose1d(hidden_dims[-1], out_channels,
                                        kernel_size=4, stride=2, padding=1)

    def forward(self, z):
        h = self.fc(z)
        h = h.view(-1, 256, 4)
        h = self.decoder(h)
        x_recon = torch.tanh(self.final(h))
        return x_recon
```

8.2 KL Divergence

```
def kl_divergence(mu, logvar):
    """
    KL(q(z|x) || p(z))
    KL = -0.5 * sum(1 + log(sigma^2) - mu^2 - sigma^2)
    """
    return -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp(), dim=-1)


def vae_loss(x, x_recon, mu, logvar, beta=1.0):
    # Reconstruction
    recon_loss = F.mse_loss(x_recon, x, reduction='none').sum(dim=[1, 2])

    # KL divergence
    kl_loss = kl_divergence(mu, logvar)

    # Total
    total_loss = recon_loss + beta * kl_loss

    return total_loss.mean(), recon_loss.mean(), kl_loss.mean()
```

8.3 Multi-Resolution STFT Loss

```python
class MultiResolutionSTFTLoss(nn.Module):
    def __init__(self, fft_sizes=[512, 1024, 2048],
                 hop_sizes=[50, 120, 240],
                 win_lengths=[240, 600, 1200]):
        super().__init__()
        self.fft_sizes = fft_sizes
        self.hop_sizes = hop_sizes
        self.win_lengths = win_lengths

    def stft(self, x, fft_size, hop_size, win_length):
        window = torch.hann_window(win_length, device=x.device)
        x = x.squeeze(1)
        stft_out = torch.stft(x, fft_size, hop_size, win_length,
                              window, return_complex=True)
        return torch.abs(stft_out)

    def forward(self, x, x_recon):
        total_loss = 0.0

        for fft_size, hop_size, win_length in zip(
            self.fft_sizes, self.hop_sizes, self.win_lengths
        ):
            x_mag = self.stft(x, fft_size, hop_size, win_length)
            x_recon_mag = self.stft(x_recon, fft_size, hop_size, win_length)

            # Spectral convergence
            sc_loss = torch.norm(x_mag - x_recon_mag, p='fro') / \
                      torch.norm(x_mag, p='fro')

            # Log magnitude
            log_loss = F.l1_loss(torch.log(x_mag + 1e-7),
                                 torch.log(x_recon_mag + 1e-7))

            total_loss += sc_loss + log_loss

        return total_loss / len(self.fft_sizes)
```

8.4 DDPM 구현

```python
class SimpleDDPM(nn.Module):
    def __init__(self, denoise_net, T=1000, beta_min=1e-4, beta_max=0.02):
        super().__init__()
        self.denoise_net = denoise_net
        self.T = T

        # Linear schedule
        betas = torch.linspace(beta_min, beta_max, T)
        alphas = 1 - betas
        alphas_cumprod = torch.cumprod(alphas, dim=0)

        self.register_buffer('betas', betas)
        self.register_buffer('alphas', alphas)
        self.register_buffer('alphas_cumprod', alphas_cumprod)
        self.register_buffer('sqrt_alphas_cumprod', torch.sqrt(alphas_cumproc
        self.register_buffer('sqrt_one_minus_alphas_cumprod',
                             torch.sqrt(1 - alphas_cumprod))

    def q_sample(self, x0, t, noise=None):
        """Forward process"""
        if noise is None:
            noise = torch.randn_like(x0)

        sqrt_alpha = self.sqrt_alphas_cumprod[t][:, None, None]
        sqrt_one_minus_alpha = self.sqrt_one_minus_alphas_cumprod[t][:, None,

        return sqrt_alpha * x0 + sqrt_one_minus_alpha * noise

    def p_losses(self, x0, t, noise=None, cond=None):
        """Training loss"""
        if noise is None:
            noise = torch.randn_like(x0)

        x_t = self.q_sample(x0, t, noise)
        noise_pred = self.denoise_net(x_t, t, cond)

        return F.mse_loss(noise_pred, noise)

    @torch.no_grad()
    def p_sample(self, x_t, t, cond=None):
        """Single denoising step"""
        betas_t = self.betas[t][:, None, None]
        sqrt_one_minus_alpha = self.sqrt_one_minus_alphas_cumprod[t][:, None,
        sqrt_recip_alpha = (1 / torch.sqrt(self.alphas[t]))[:, None, None]

        noise_pred = self.denoise_net(x_t, t, cond)

        model_mean = sqrt_recip_alpha * (
            x_t - betas_t * noise_pred / sqrt_one_minus_alpha
        )

        if t[0] > 0:
            noise = torch.randn_like(x_t)
            return model_mean + torch.sqrt(betas_t) * noise
        else:
            return model_mean
```

```
@torch.no_grad()
def sample(self, shape, cond=None, device='cuda'):
    """Full sampling loop"""
    x = torch.randn(shape, device=device)

    for t in reversed(range(self.T)):
        t_batch = torch.full((shape[0],), t, device=device, dtype=torch.l
        x = self.p_sample(x, t_batch, cond)

    return x
```

8.5 DDIM 샘플러

```python
class DDIMSampler:
    def __init__(self, ddpm_model, ddim_steps=50, eta=0.0):
        self.model = ddpm_model
        self.ddim_steps = ddim_steps
        self.eta = eta

        # Timestep subsequence
        c = ddpm_model.T // ddim_steps
        self.timesteps = list(range(0, ddpm_model.T, c))

    @torch.no_grad()
    def sample(self, shape, cond=None, device='cuda'):
        x = torch.randn(shape, device=device)

        timesteps = list(reversed(self.timesteps))

        for i, t in enumerate(timesteps):
            t_batch = torch.full((shape[0],), t, device=device, dtype=torch.l

            noise_pred = self.model.denoise_net(x, t_batch, cond)

            alpha_cumprod_t = self.model.alphas_cumprod[t]

            if i < len(timesteps) - 1:
                alpha_cumprod_prev = self.model.alphas_cumprod[timesteps[i +
            else:
                alpha_cumprod_prev = torch.tensor(1.0, device=device)

            # Predict x0
            x0_pred = (x - torch.sqrt(1 - alpha_cumprod_t) * noise_pred) / \
                      torch.sqrt(alpha_cumprod_t)

            # Sigma
            sigma = self.eta * torch.sqrt(
                (1 - alpha_cumprod_prev) / (1 - alpha_cumprod_t) *
                (1 - alpha_cumprod_t / alpha_cumprod_prev)
            )

            # Direction
            dir_xt = torch.sqrt(1 - alpha_cumprod_prev - sigma**2) * noise_pr

            # x_{t-1}
            noise = torch.randn_like(x) if i < len(timesteps) - 1 else 0
            x = torch.sqrt(alpha_cumprod_prev) * x0_pred + dir_xt + sigma * r

        return x
```

---

9. 하이퍼파라미터 가이드

9.1 VAE 학습

하이퍼파라미터	권장값	설명

`beta` (KL 가중치)	0.001 ~ 0.1	오디오는 낮게 (복원 우선)
`learning_rate`	1e-4 ~ 3e-4	AdamW
`batch_size`	16 ~ 64	GPU 메모리에 따라
`latent_dim`	32 ~ 128	압축률/품질 트레이드오프
`KL annealing`	0 → 1 over 10K steps	Posterior collapse 방지

9.2 Diffusion 학습

하이퍼파라미터	권장값	설명
`T` (타임스텝)	1000	표준값
`beta_min`	1e-4	노이즈 스케줄 시작
`beta_max`	0.02	노이즈 스케줄 끝
`learning_rate`	1e-4	AdamW
`EMA decay`	0.9999	Exponential Moving Average
`CFG dropout`	0.1	조건 드롭 확률
`CFG scale`	3.0 ~ 7.0	추론 시 가이던스 강도

9.3 DDIM 샘플링

하이퍼파라미터	권장값	설명
`ddim_steps`	50 ~ 100	20배 가속
`eta`	0.0	결정적 (재현성)
`eta`	0.5 ~ 1.0	확률적 (다양성)

---

10. 참고 문헌

1. **VAE**: Kingma & Welling, "Auto-Encoding Variational Bayes", ICLR 2014
2. **DDPM**: Ho et al., "Denoising Diffusion Probabilistic Models", NeurIPS 2020
3. **DDIM**: Song et al., "Denoising Diffusion Implicit Models", ICLR 2021
4. **CFG**: Ho & Salimans, "Classifier-Free Diffusion Guidance", NeurIPS Workshop 2021
5. **EnCodec**: Défossez et al., "High Fidelity Neural Audio Compression", TMLR 2023
6. **DiT**: Peebles & Xie, "Scalable Diffusion Models with Transformers", ICCV 2023
7. **Stable Audio**: Evans et al., "Stable Audio: Fast Timing-Conditioned Latent Audio Diffusion", 2024
8. **DiffRhythm**: Liu et al., "DiffRhythm: Blazingly Fast End-to-End Song Generation", 2025

다음 파트: Part 2. 로봇 AI 수학적 심화